

AI & AC

ASSIGNMENT-11.1

BATCH:28

HT NO.:2303A51690

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

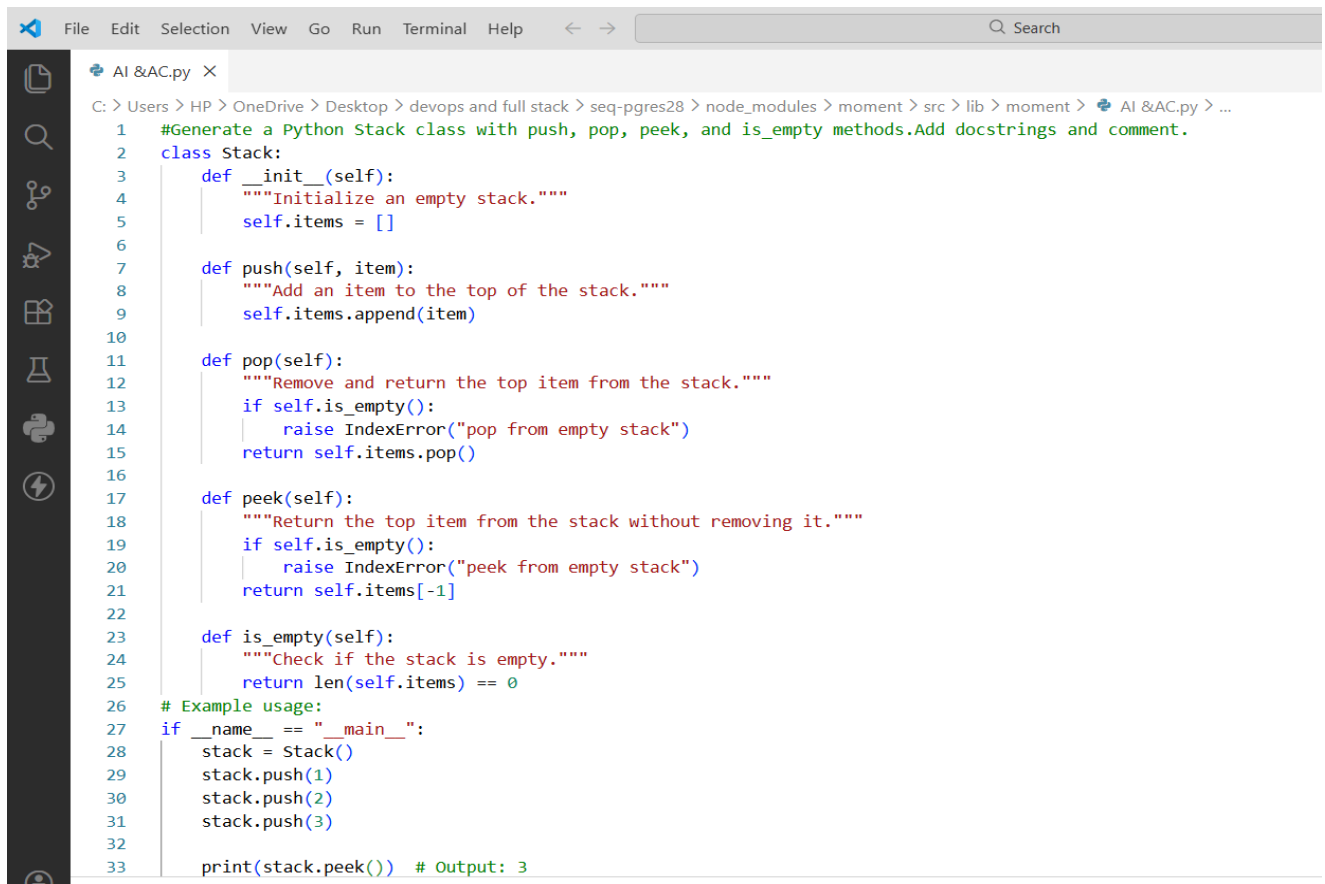
- A functional stack implementation with all required methods and docstrings.

PROMPT:

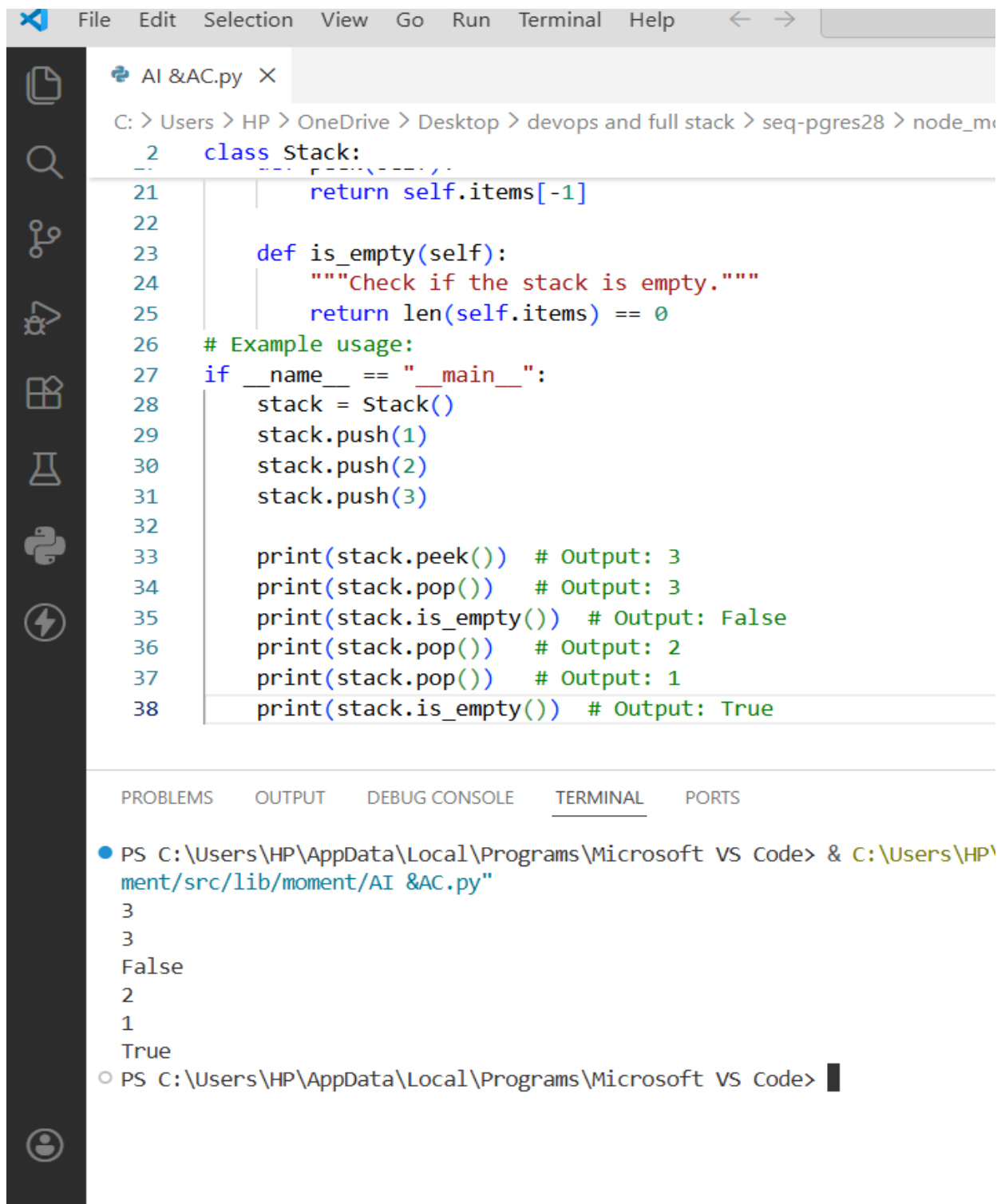
Generate a Python Stack class with push, pop, peek, and is_empty methods.

Add docstrings and comments.

CODE AND OUTPUT:

The image shows a screenshot of the Visual Studio Code (VS Code) editor. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. A search bar is located on the right side of the menu bar. The left sidebar contains a vertical toolbar with icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, Remote Explorer, and a user profile icon. The main editor area displays a Python file named 'AI &AC.py'. The file's content is a Python class named 'Stack' with methods for initialization, pushing, popping, peeking, and checking if the stack is empty. It also includes an example usage section at the bottom. The file path shown in the breadcrumb is 'C:\> Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...'.

```
1  #Generate a Python Stack class with push, pop, peek, and is_empty methods.Add docstrings and comment.
2  class Stack:
3      def __init__(self):
4          """Initialize an empty stack."""
5          self.items = []
6
7      def push(self, item):
8          """Add an item to the top of the stack."""
9          self.items.append(item)
10
11     def pop(self):
12         """Remove and return the top item from the stack."""
13         if self.is_empty():
14             raise IndexError("pop from empty stack")
15         return self.items.pop()
16
17     def peek(self):
18         """Return the top item from the stack without removing it."""
19         if self.is_empty():
20             raise IndexError("peek from empty stack")
21         return self.items[-1]
22
23     def is_empty(self):
24         """Check if the stack is empty."""
25         return len(self.items) == 0
26
27     # Example usage:
28     if __name__ == "__main__":
29         stack = Stack()
30         stack.push(1)
31         stack.push(2)
32         stack.push(3)
33
34         print(stack.peek()) # Output: 3
```



The image shows a VS Code editor window with a file named 'AI & AC.py'. The code defines a 'Stack' class with methods 'push', 'pop', and 'is_empty'. It also includes an example usage in the 'if __name__ == "__main__":' block. The terminal at the bottom shows the command to run the script and its output, which matches the comments in the code.

```
File Edit Selection View Go Run Terminal Help
AI & AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_m
2 class Stack:
21     return self.items[-1]
22
23     def is_empty(self):
24         """Check if the stack is empty."""
25         return len(self.items) == 0
26 # Example usage:
27 if __name__ == "__main__":
28     stack = Stack()
29     stack.push(1)
30     stack.push(2)
31     stack.push(3)
32
33     print(stack.peek()) # Output: 3
34     print(stack.pop()) # Output: 3
35     print(stack.is_empty()) # Output: False
36     print(stack.pop()) # Output: 2
37     print(stack.pop()) # Output: 1
38     print(stack.is_empty()) # Output: True

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\ment/src/lib/moment/AI & AC.py"
3
3
False
2
1
True
○ PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> █
```

JUSTIFICATION:

1. Follows LIFO (Last In First Out) principle.
2. Useful for function calls, recursion, and undo operations.
3. push() and pop() operations work in $O(1)$ time.

4. Simple implementation using Python list.
5. Efficient memory usage for temporary data storage.
6. Helps in expression evaluation and syntax parsing.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

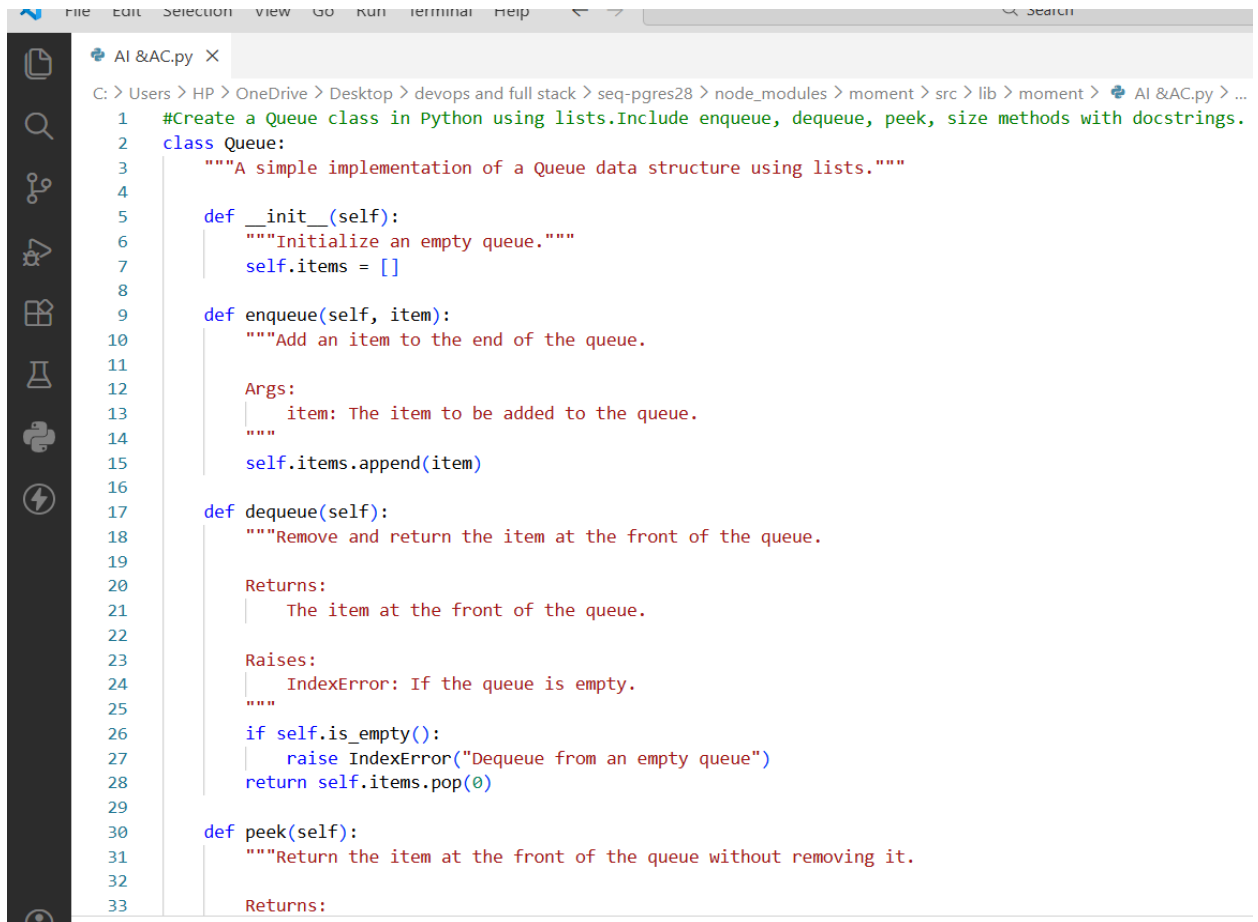
Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

PROMPT:

Create a Queue class in Python using lists. Include enqueue, dequeue, peek, size methods with docstrings.

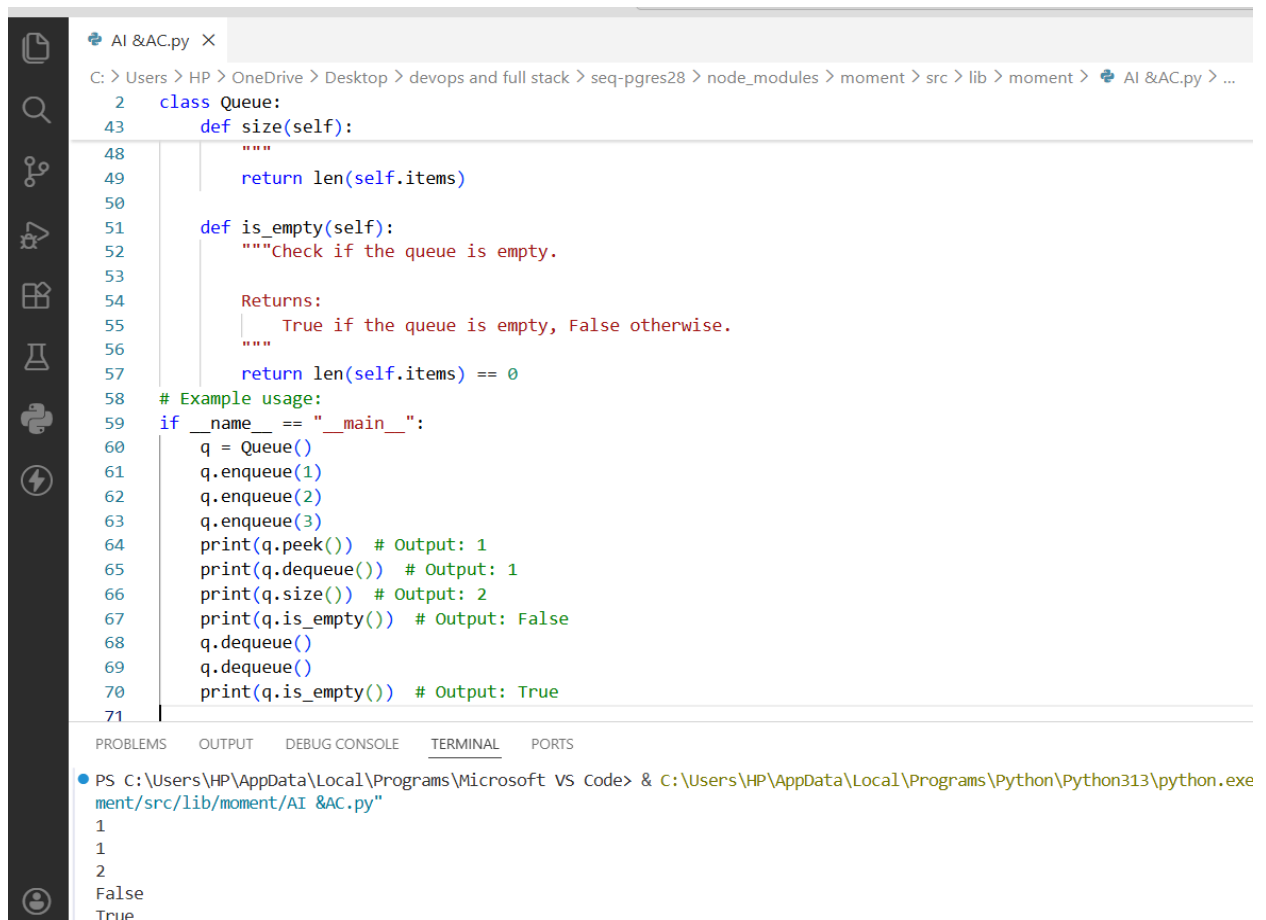
CODE & OUTPUT:



The image shows a screenshot of the Visual Studio Code (VS Code) editor. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. A search bar is visible on the right. The left sidebar contains a vertical toolbar with icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, Python, and a power icon. The main editor area displays a Python file named 'AI &AC.py'. The file path is shown as 'C:\> Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...'. The code defines a 'Queue' class with methods for enqueue, dequeue, and peek, each with detailed docstrings.

```
1  #Create a Queue class in Python using lists.Include enqueue, dequeue, peek, size methods with docstrings.
2  class Queue:
3      """A simple implementation of a Queue data structure using lists."""
4
5      def __init__(self):
6          """Initialize an empty queue."""
7          self.items = []
8
9      def enqueue(self, item):
10         """Add an item to the end of the queue.
11
12         Args:
13             item: The item to be added to the queue.
14         """
15         self.items.append(item)
16
17     def dequeue(self):
18         """Remove and return the item at the front of the queue.
19
20         Returns:
21             The item at the front of the queue.
22
23         Raises:
24             IndexError: If the queue is empty.
25         """
26         if self.is_empty():
27             raise IndexError("Dequeue from an empty queue")
28         return self.items.pop(0)
29
30     def peek(self):
31         """Return the item at the front of the queue without removing it.
32
33         Returns:
```

```
File Edit Selection View Go Run Terminal Help Search
AI & AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > .
2 class Queue:
29
30 def peek(self):
31     """Return the item at the front of the queue without removing it.
32
33     Returns:
34         The item at the front of the queue.
35
36     Raises:
37         IndexError: If the queue is empty.
38     """
39     if self.is_empty():
40         raise IndexError("Peek from an empty queue")
41     return self.items[0]
42
43 def size(self):
44     """Return the number of items in the queue.
45
46     Returns:
47         The number of items in the queue.
48     """
49     return len(self.items)
50
51 def is_empty(self):
52     """Check if the queue is empty.
53
54     Returns:
55         True if the queue is empty, False otherwise.
56     """
57     return len(self.items) == 0
58 # Example usage:
59 if __name__ == "__main__":
60     q = Queue()
```



```
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
2 class Queue:
43     def size(self):
48         """
49         return len(self.items)
50
51     def is_empty(self):
52         """check if the queue is empty.
53
54         Returns:
55             True if the queue is empty, False otherwise.
56         """
57         return len(self.items) == 0
58 # Example usage:
59 if __name__ == "__main__":
60     q = Queue()
61     q.enqueue(1)
62     q.enqueue(2)
63     q.enqueue(3)
64     print(q.peak()) # Output: 1
65     print(q.dequeue()) # Output: 1
66     print(q.size()) # Output: 2
67     print(q.is_empty()) # Output: False
68     q.dequeue()
69     q.dequeue()
70     print(q.is_empty()) # Output: True
71
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe
ment/src/lib/moment/AI &AC.py"
1
1
2
False
True
```

JUSTIFICATION:

1. Follows FIFO (First In First Out) principle.
2. Suitable for scheduling and service systems.
3. Maintains order of data insertion.
4. Efficient enqueue and dequeue operations.
5. Used in task scheduling and buffering systems.
6. Simple and easy to implement using lists.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
pass
```

```
class LinkedList:
```

pass

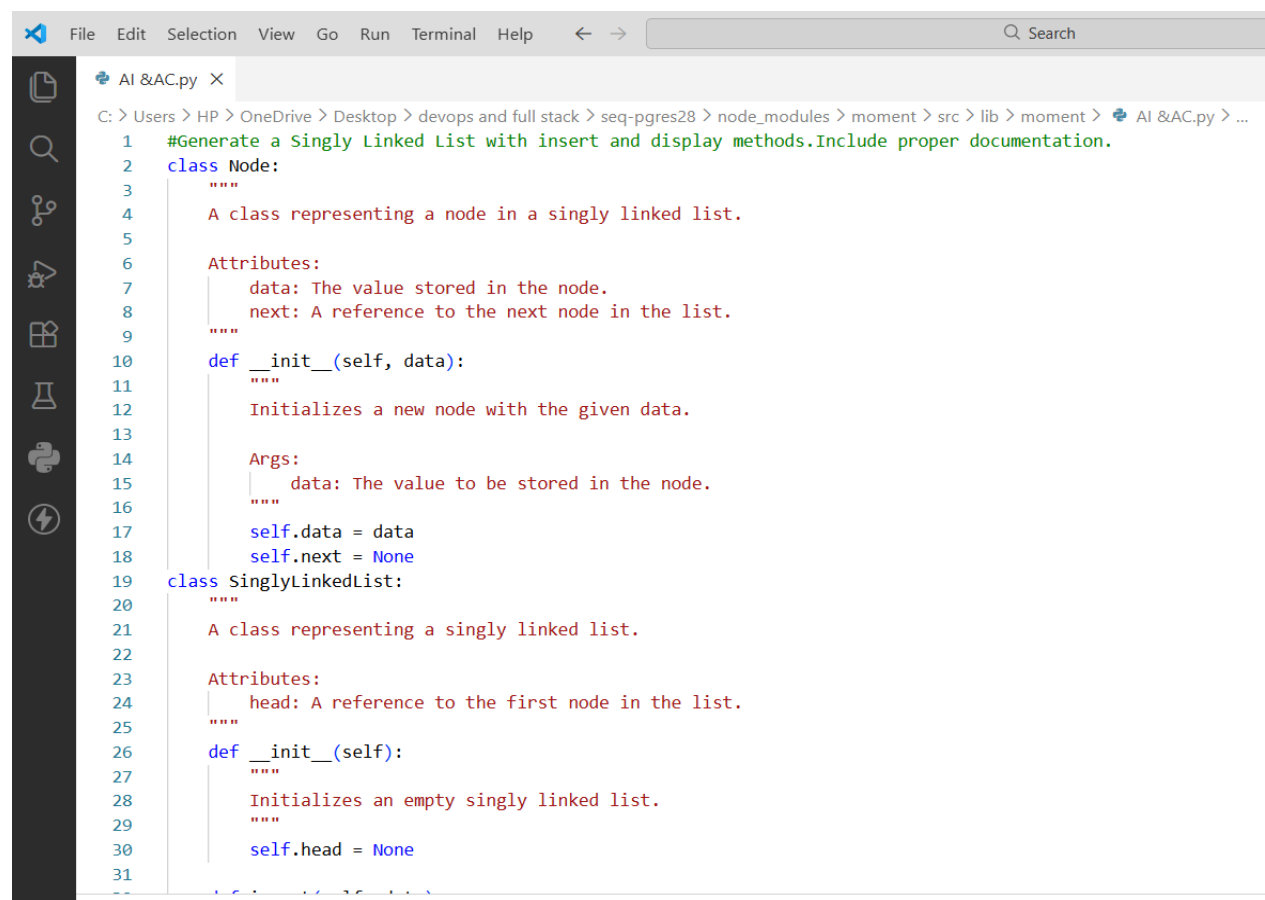
Expected Output:

- A working linked list implementation with clear method documentation.

PROMPT:

Generate a Singly Linked List with insert and display methods. Include proper documentation.

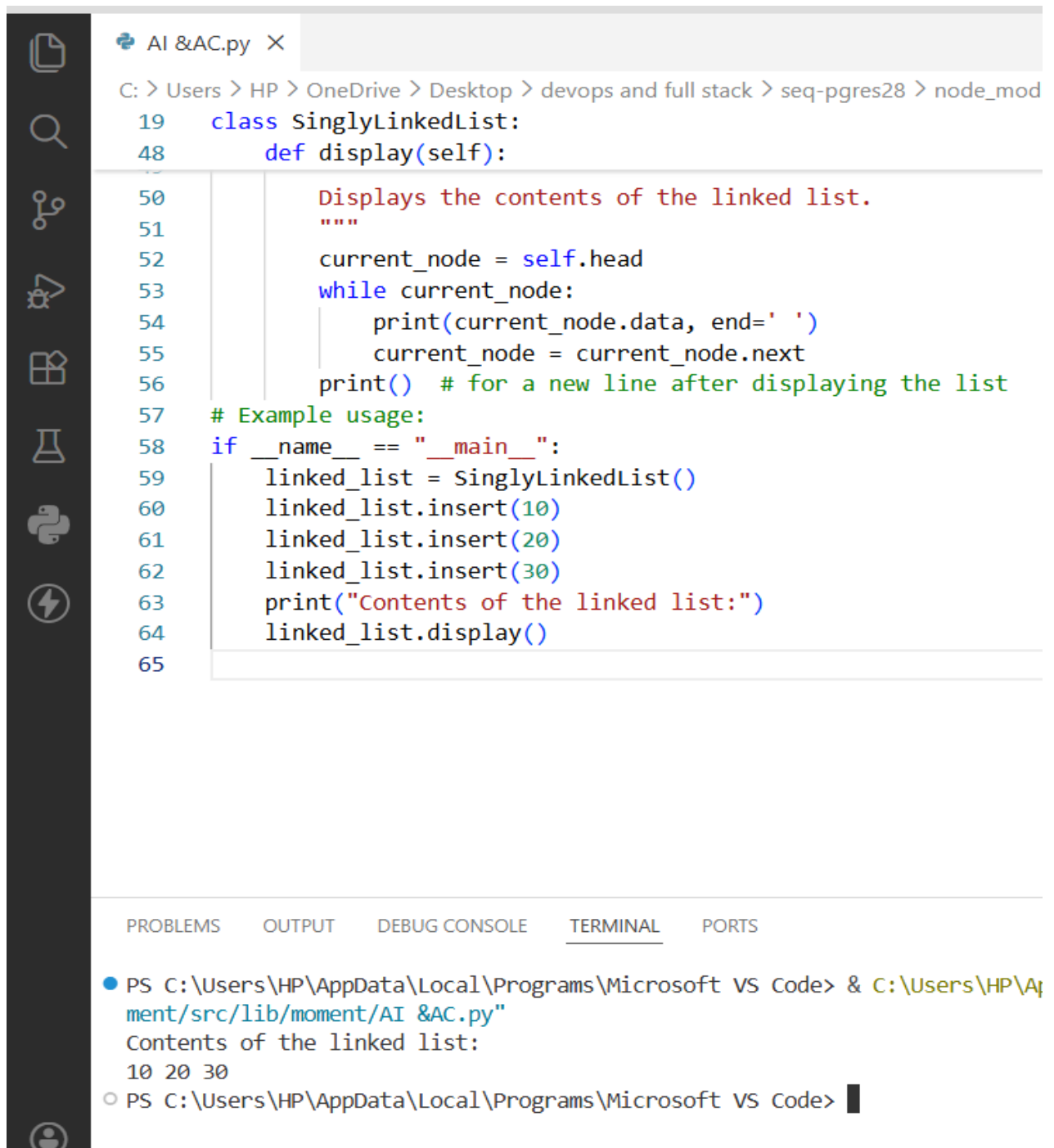
CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help < -> Search
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
1 #Generate a Singly Linked List with insert and display methods.Include proper documentation.
2 class Node:
3     """
4     A class representing a node in a singly linked list.
5
6     Attributes:
7         data: The value stored in the node.
8         next: A reference to the next node in the list.
9     """
10    def __init__(self, data):
11        """
12        Initializes a new node with the given data.
13
14        Args:
15            data: The value to be stored in the node.
16        """
17        self.data = data
18        self.next = None
19    class SinglyLinkedList:
20        """
21        A class representing a singly linked list.
22
23        Attributes:
24            head: A reference to the first node in the list.
25        """
26        def __init__(self):
27            """
28            Initializes an empty singly linked list.
29            """
30            self.head = None
31
```



```
File Edit Selection View Go Run Terminal Help Search
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
19 class SinglyLinkedList:
26     def __init__(self):
30         self.head = None
31
32     def insert(self, data):
33         """
34         Inserts a new node with the given data at the end of the list.
35
36         Args:
37         | data: The value to be stored in the new node.
38         """
39         new_node = Node(data)
40         if self.head is None:
41             self.head = new_node
42             return
43         last_node = self.head
44         while last_node.next:
45             last_node = last_node.next
46         last_node.next = new_node
47
48     def display(self):
49         """
50         Displays the contents of the linked list.
51         """
52         current_node = self.head
53         while current_node:
54             print(current_node.data, end=' ')
55             current_node = current_node.next
56         print() # for a new line after displaying the list
57 # Example usage:
58 if __name__ == "__main__":
59     linked_list = SinglyLinkedList()
```



The image shows a screenshot of a Visual Studio Code editor window. The top part of the window displays a Python file named `AI &AC.py`. The code defines a `SinglyLinkedList` class with a `display` method and includes example usage in the `__main__` block. The bottom part of the window shows the `TERMINAL` output, which displays the contents of the linked list as `10 20 30`.

```
19 class SinglyLinkedList:
48     def display(self):
50         """
51         Displays the contents of the linked list.
52         """
53         current_node = self.head
54         while current_node:
55             print(current_node.data, end=' ')
56             current_node = current_node.next
57         print() # for a new line after displaying the list
58 # Example usage:
59 if __name__ == "__main__":
60     linked_list = SinglyLinkedList()
61     linked_list.insert(10)
62     linked_list.insert(20)
63     linked_list.insert(30)
64     print("Contents of the linked list:")
65     linked_list.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Microsoft VS Code\bin\python.exe C:\Users\HP\AppData\Local\Programs\Microsoft VS Code\src\lib\moment\AI &AC.py
Contents of the linked list:
10 20 30
○ PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> █
```

JUSTIFICATION:

1. Dynamic memory allocation (no fixed size).
2. Efficient insertion and deletion compared to arrays.
3. Does not require contiguous memory.
4. Suitable for frequently changing data.
5. Flexible structure for building other data structures.

6. Useful in implementing stacks, queues, and graphs.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
pass
```

Expected Output:

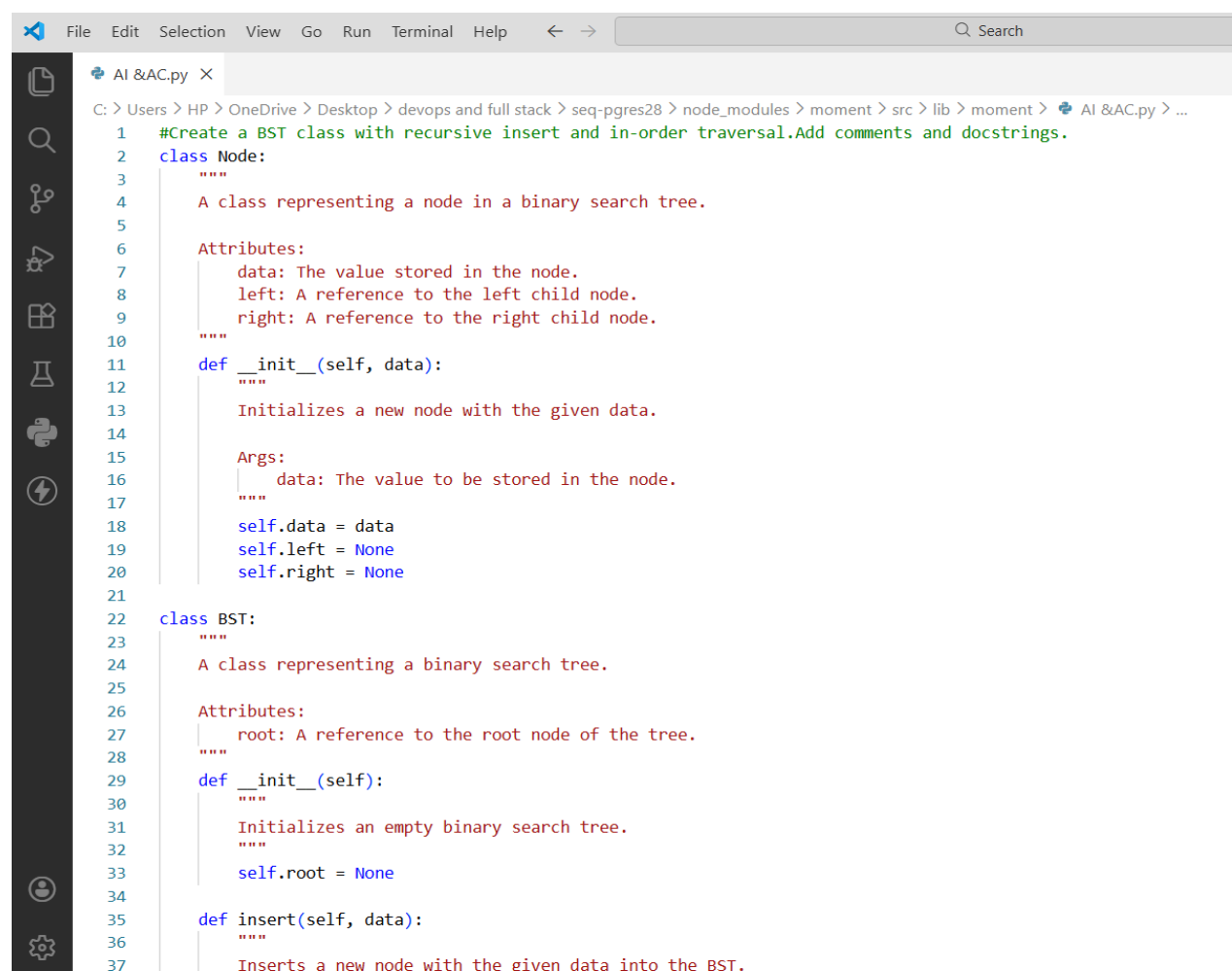
- BST implementation with recursive insert and traversal methods.

PROMPT:

Create a BST class with recursive insert and in-order traversal.

Add comments and docstrings.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help  Search
AI &AC.py X
C:\Users\HP\OneDrive\Desktop\devops and full stack\seq-pgres28\node_modules\moment\src\lib\moment> AI &AC.py > ...
1  #Create a BST class with recursive insert and in-order traversal.Add comments and docstrings.
2  class Node:
3      """
4      A class representing a node in a binary search tree.
5
6      Attributes:
7      data: The value stored in the node.
8      left: A reference to the left child node.
9      right: A reference to the right child node.
10     """
11     def __init__(self, data):
12         """
13         Initializes a new node with the given data.
14
15         Args:
16         data: The value to be stored in the node.
17         """
18         self.data = data
19         self.left = None
20         self.right = None
21
22     class BST:
23         """
24         A class representing a binary search tree.
25
26         Attributes:
27         root: A reference to the root node of the tree.
28         """
29         def __init__(self):
30             """
31             Initializes an empty binary search tree.
32             """
33             self.root = None
34
35         def insert(self, data):
36             """
37             Inserts a new node with the given data into the BST.
```

```
File Edit Selection View Go Run Terminal Help < -> Search
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
22 class BST:
35     def insert(self, data):
36         """
37         Inserts a new node with the given data into the BST.
38
39         Args:
40             data: The value to be inserted into the BST.
41         """
42         if self.root is None:
43             self.root = Node(data)
44             return
45         self._insert_recursive(self.root, data)
46
47     def _insert_recursive(self, current_node, data):
48         """
49         Recursively inserts a new node with the given data into the BST.
50
51         Args:
52             current_node: The current node being examined during insertion.
53             data: The value to be inserted into the BST.
54         """
55         if data < current_node.data:
56             if current_node.left is None:
57                 current_node.left = Node(data)
58             else:
59                 self._insert_recursive(current_node.left, data)
60         elif data > current_node.data:
61             if current_node.right is None:
62                 current_node.right = Node(data)
63             else:
64                 self._insert_recursive(current_node.right, data)
65
66     def inorder_traversal(self):
67         """
68         Performs an in-order traversal of the BST and returns a list of values in sorted order.
69
70         Returns:
```

```
22 class BST:
66     def inorder_traversal(self):
71         """
72         A list containing values in sorted order from the BST.
73         """
74         result = []
75         self._inorder_recursive(self.root, result)
76         return result
77     def _inorder_recursive(self, current_node, result):
78         """
79         Recursively performs an in-order traversal of the BST and appends values to result list.
80         Args:
81         current_node: The current node being examined during traversal.
82         result: The list to which values are appended during traversal.
83         """
84         if current_node is not None:
85             self._inorder_recursive(current_node.left, result)
86             result.append(current_node.data)
87             self._inorder_recursive(current_node.right, result)
88     # Example usage:
89     if __name__ == "__main__":
90         bst = BST()
91         bst.insert(5)
92         bst.insert(3)
93         bst.insert(7)
94         bst.insert(2)
95         bst.insert(4)
96         bst.insert(6)
97         bst.insert(8)
98         print("In-order Traversal:", bst.inorder_traversal())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

In-order Traversal: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>

JUSTIFICATION:

- 1.Maintains sorted order of elements.
- 2.Enables efficient searching in **$O(\log n)$** (average case).
- 3.Supports recursive insertion and traversal.
- 4.In-order traversal gives sorted output.
- 5.Efficient for hierarchical data representation.
- 6.Used in databases and search applications.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

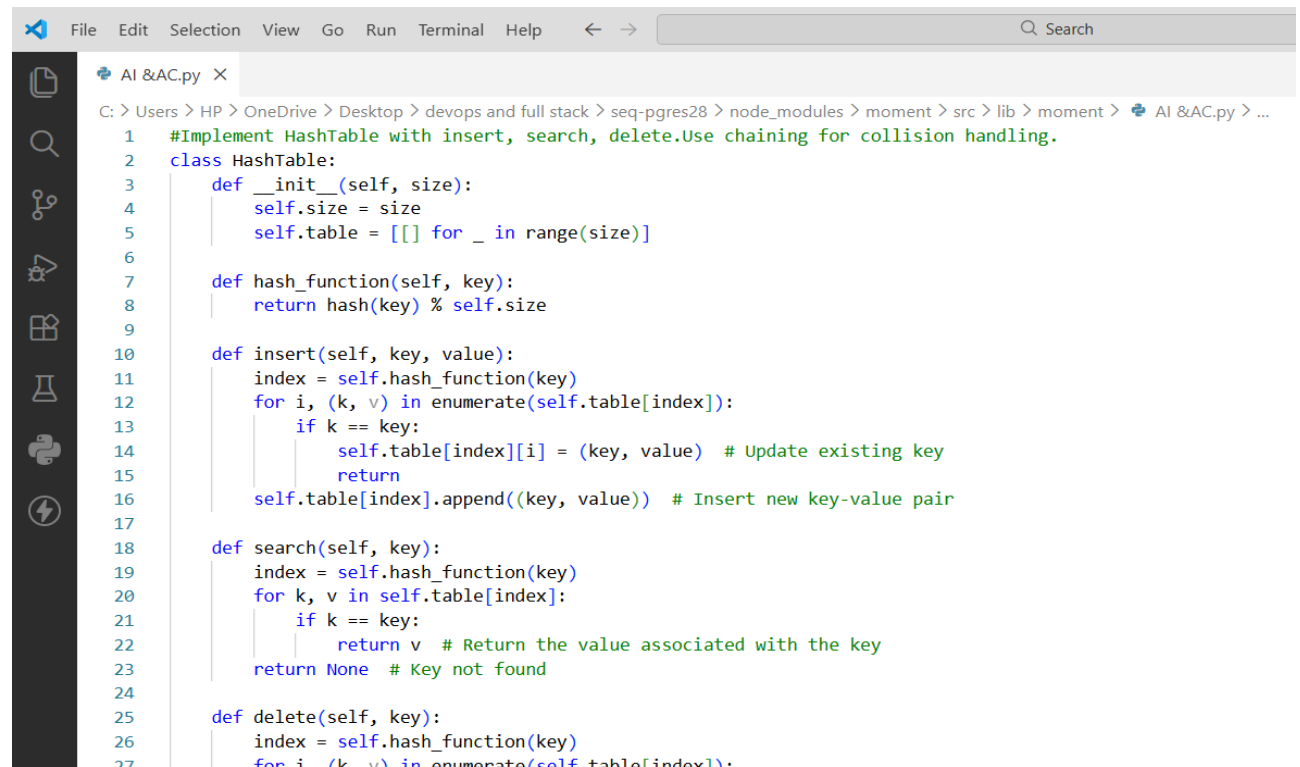
Expected Output:

- Collision handling using chaining, with well-commented methods.

PROMPT:

Implement HashTable with insert, search, delete. Use chaining for collision handling.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help ← → Search
AI & AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > ...
1 #Implement HashTable with insert, search, delete. Use chaining for collision handling.
2 class HashTable:
3     def __init__(self, size):
4         self.size = size
5         self.table = [[] for _ in range(size)]
6
7     def hash_function(self, key):
8         return hash(key) % self.size
9
10    def insert(self, key, value):
11        index = self.hash_function(key)
12        for i, (k, v) in enumerate(self.table[index]):
13            if k == key:
14                self.table[index][i] = (key, value) # Update existing key
15                return
16        self.table[index].append((key, value)) # Insert new key-value pair
17
18    def search(self, key):
19        index = self.hash_function(key)
20        for k, v in self.table[index]:
21            if k == key:
22                return v # Return the value associated with the key
23        return None # Key not found
24
25    def delete(self, key):
26        index = self.hash_function(key)
27        for i, (k, v) in enumerate(self.table[index]):
```

```
File Edit Selection View Go Run Terminal Help
AI & AC.py X
C:\Users\HP> HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > ...
2 class HashTable:
18 def search(self, key):
21     if k == key:
22         return v # Return the value associated with the key
23     return None # Key not found
24
25 def delete(self, key):
26     index = self.hash_function(key)
27     for i, (k, v) in enumerate(self.table[index]):
28         if k == key:
29             del self.table[index][i] # Remove the key-value pair
30             return True # Deletion successful
31     return False # Key not found
32
33 # Example usage
34 hash_table = HashTable(10)
35 hash_table.insert("name", "Alice")
36 hash_table.insert("age", 30)
37 print(hash_table.search("name")) # Output: Alice
38 print(hash_table.search("age")) # Output: 30
39 hash_table.delete("name")
40 print(hash_table.search("name")) # Output: None

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/HP/Desktop/devops and full stack/seq-pgres28/node_modules/moment/src/lib/moment/AI & AC.py"
Alice
30
None
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> 
```

JUSTIFICATION:

1. Provides **O(1)** average search time.
2. Efficient key-value storage.
3. Collision handling improves reliability.
4. Fast insertion and deletion.
5. Ideal for indexing and caching systems.
6. Scales well for large datasets.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

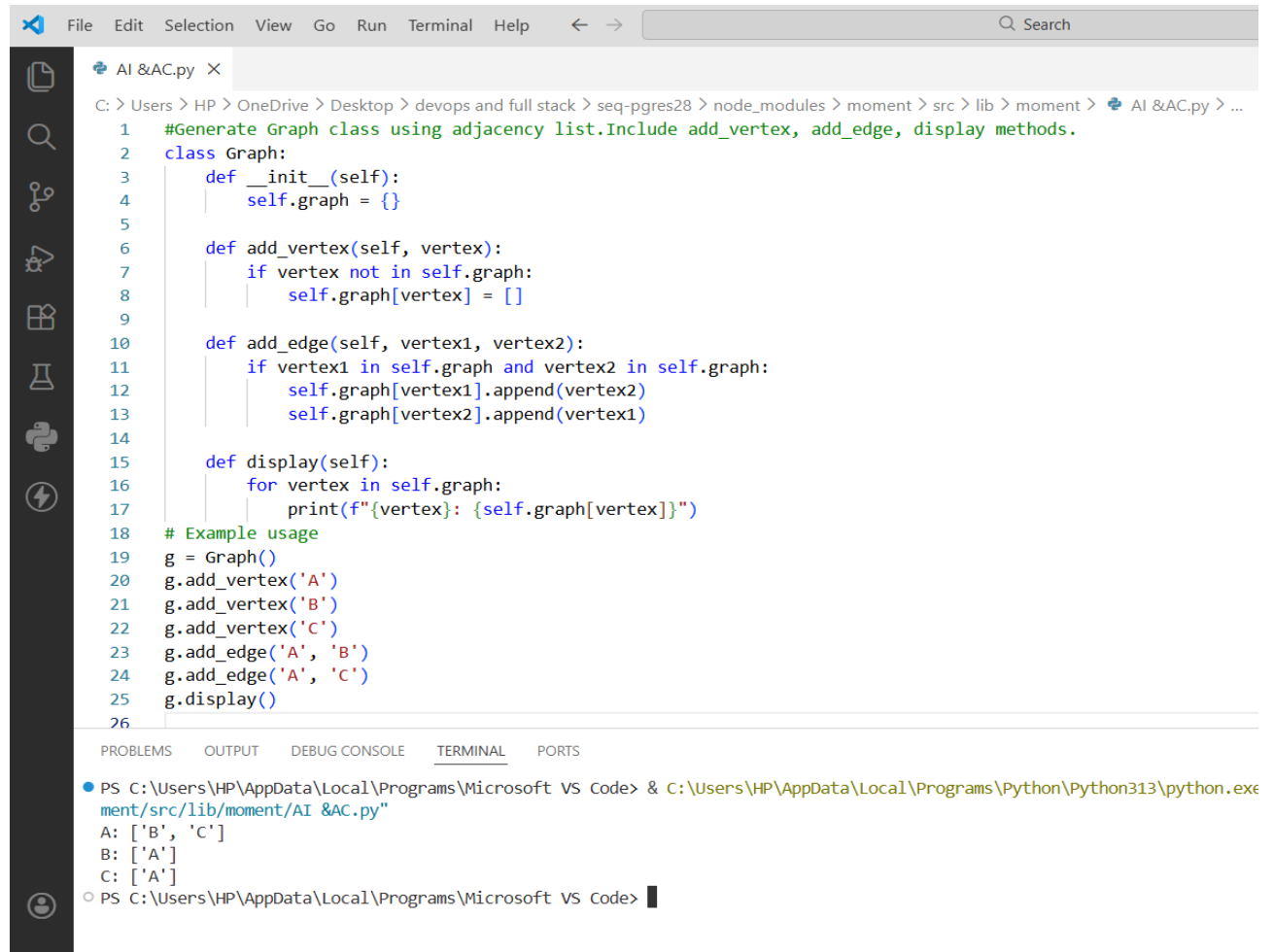
Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

PROMPT:

Generate Graph class using adjacency list. Include add_vertex, add_edge, display methods.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help
AI & AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > ...
1 #Generate Graph class using adjacency list.Include add_vertex, add_edge, display methods.
2 class Graph:
3     def __init__(self):
4         self.graph = {}
5
6     def add_vertex(self, vertex):
7         if vertex not in self.graph:
8             self.graph[vertex] = []
9
10    def add_edge(self, vertex1, vertex2):
11        if vertex1 in self.graph and vertex2 in self.graph:
12            self.graph[vertex1].append(vertex2)
13            self.graph[vertex2].append(vertex1)
14
15    def display(self):
16        for vertex in self.graph:
17            print(f"{vertex}: {self.graph[vertex]}")
18
19 # Example usage
20 g = Graph()
21 g.add_vertex('A')
22 g.add_vertex('B')
23 g.add_vertex('C')
24 g.add_edge('A', 'B')
25 g.add_edge('A', 'C')
26 g.display()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe
moment/src/lib/moment/AI & AC.py"
A: ['B', 'C']
B: ['A']
C: ['A']
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Represents relationships between entities.
2. Efficient for sparse connections.
3. Supports path-finding algorithms.
4. Easy to add vertices and edges.
5. Models real-world networks naturally.
6. Used in routing and social networks.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's `heapq` module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

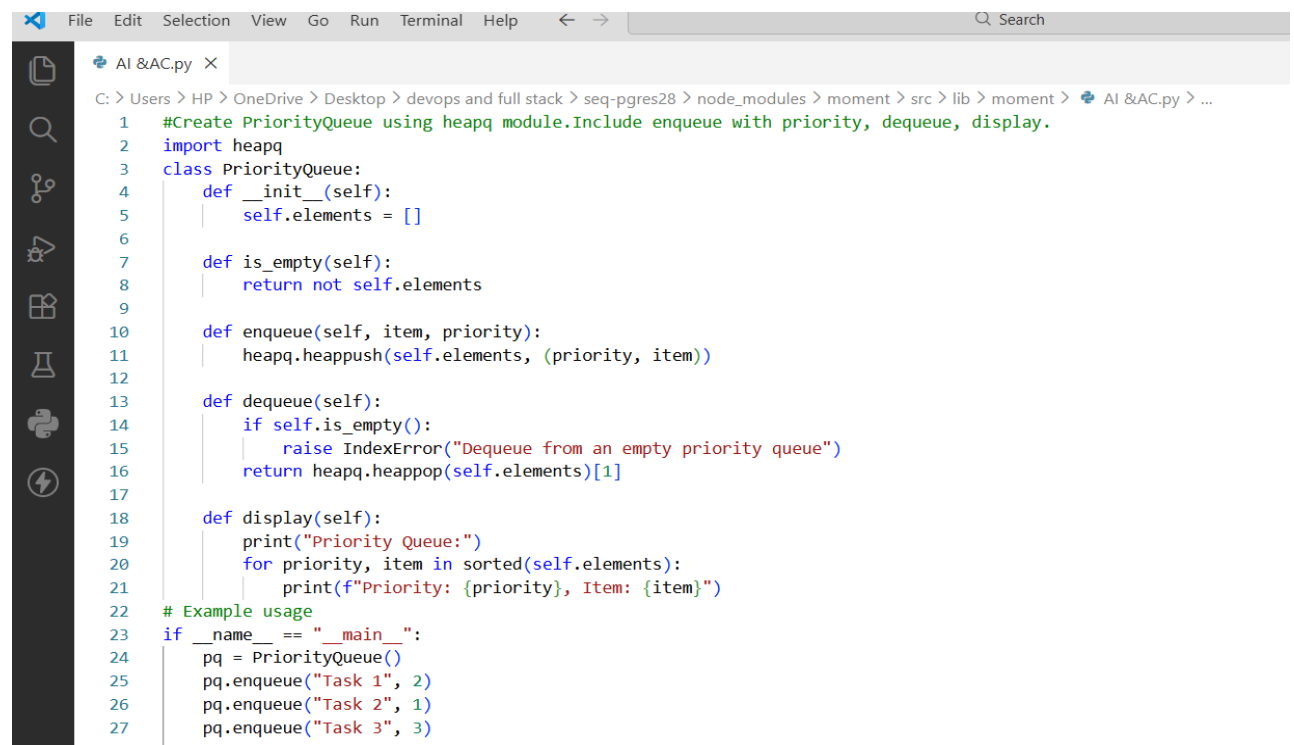
Expected Output:

- Implementation with `enqueue` (priority), `dequeue` (highest priority), and `display` methods.

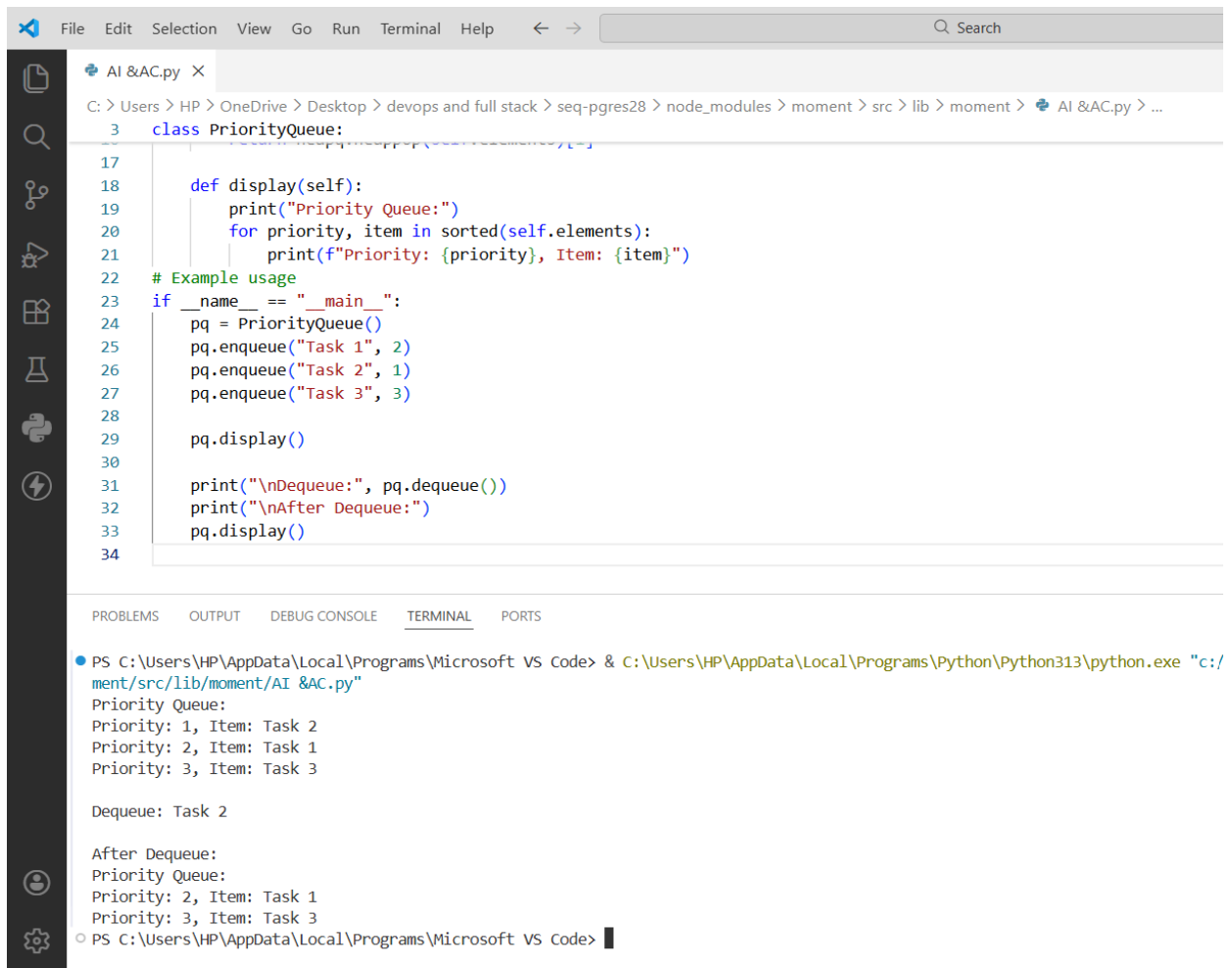
PROMPT:

Create `PriorityQueue` using `heapq` module. Include `enqueue` with priority, `dequeue`, `display`.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help ← → Search
AI & AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > ...
1 #Create PriorityQueue using heapq module.Include enqueue with priority, dequeue, display.
2 import heapq
3 class PriorityQueue:
4     def __init__(self):
5         self.elements = []
6
7     def is_empty(self):
8         return not self.elements
9
10    def enqueue(self, item, priority):
11        heapq.heappush(self.elements, (priority, item))
12
13    def dequeue(self):
14        if self.is_empty():
15            raise IndexError("Dequeue from an empty priority queue")
16        return heapq.heappop(self.elements)[1]
17
18    def display(self):
19        print("Priority Queue:")
20        for priority, item in sorted(self.elements):
21            print(f"Priority: {priority}, Item: {item}")
22
23    # Example usage
24    if __name__ == "__main__":
25        pq = PriorityQueue()
26        pq.enqueue("Task 1", 2)
27        pq.enqueue("Task 2", 1)
28        pq.enqueue("Task 3", 3)
```



```
File Edit Selection View Go Run Terminal Help
AI & AC.py
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI & AC.py > ...
3 class PriorityQueue:
17
18     def display(self):
19         print("Priority Queue:")
20         for priority, item in sorted(self.elements):
21             print(f"Priority: {priority}, Item: {item}")
22 # Example usage
23 if __name__ == "__main__":
24     pq = PriorityQueue()
25     pq.enqueue("Task 1", 2)
26     pq.enqueue("Task 2", 1)
27     pq.enqueue("Task 3", 3)
28
29     pq.display()
30
31     print("\nDequeue:", pq.dequeue())
32     print("\nAfter Dequeue:")
33     pq.display()
34
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "c:/
ment/src/lib/moment/AI & AC.py"
Priority Queue:
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3

Dequeue: Task 2

After Dequeue:
Priority Queue:
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Elements are processed based on priority.
2. Uses heap for efficient retrieval.
3. enqueue() and dequeue() are efficient.
4. Suitable for scheduling systems.
5. Used in shortest path algorithms (Dijkstra).
6. Ensures highest priority served first.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS:
```

```
pass
```

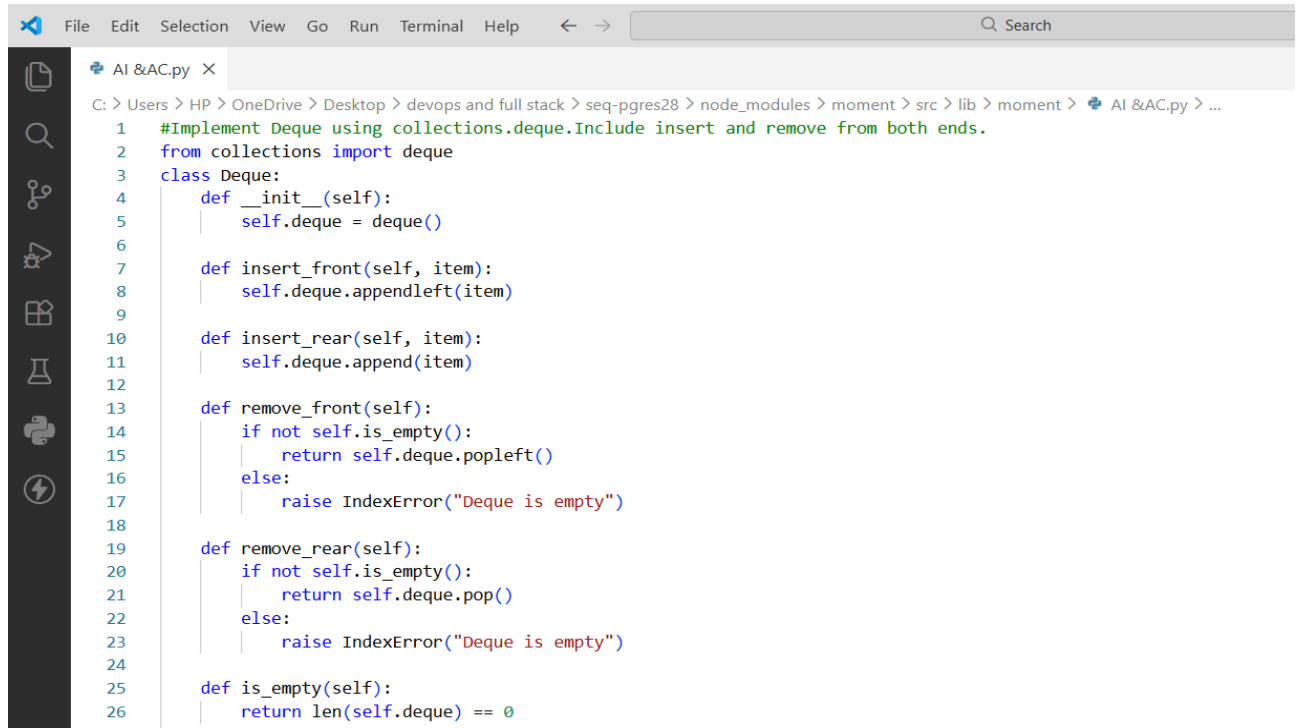
Expected Output:

- Insert and remove from both ends with docstrings.

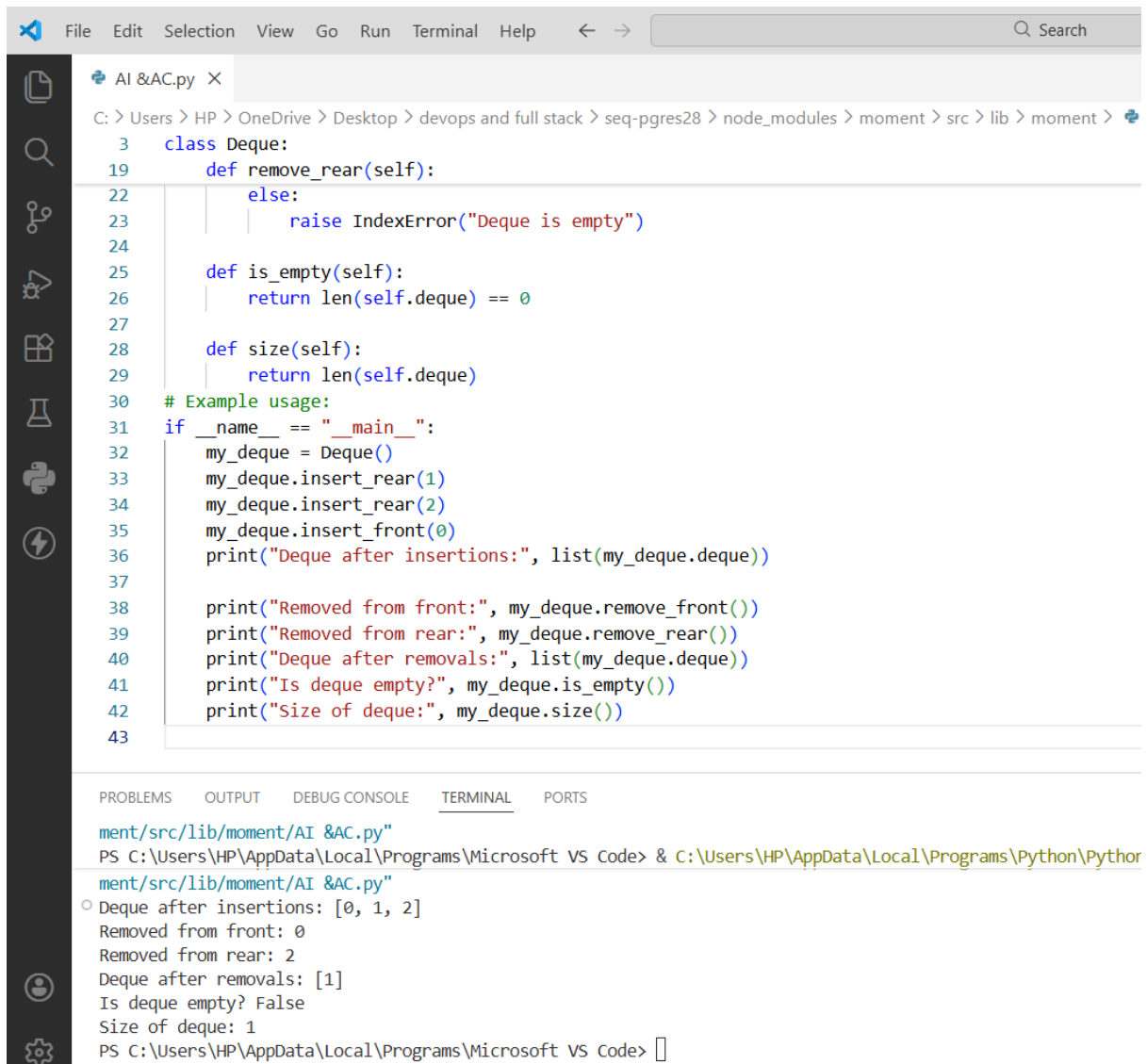
PROMPT:

Implement Deque using collections.deque. Include insert and remove from both ends.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help  Search
AI &AC.py X
C:\Users\HP> OneDrive\Desktop> devops and full stack> seq-pgres28> node_modules> moment> src> lib> moment> AI &AC.py> ...
1  #Implement Deque using collections.deque.Include insert and remove from both ends.
2  from collections import deque
3  class Deque:
4      def __init__(self):
5          self.deque = deque()
6
7      def insert_front(self, item):
8          self.deque.appendleft(item)
9
10     def insert_rear(self, item):
11         self.deque.append(item)
12
13     def remove_front(self):
14         if not self.is_empty():
15             return self.deque.popleft()
16         else:
17             raise IndexError("Deque is empty")
18
19     def remove_rear(self):
20         if not self.is_empty():
21             return self.deque.pop()
22         else:
23             raise IndexError("Deque is empty")
24
25     def is_empty(self):
26         return len(self.deque) == 0
27
```



The image shows a Visual Studio Code editor window with a Python file named `AI &AC.py`. The code defines a `Deque` class with methods for adding and removing elements from both ends, checking if it's empty, and getting its size. A main block demonstrates the usage of the deque.

```
3 class Deque:
19     def remove_rear(self):
22         else:
23             raise IndexError("Deque is empty")
24
25     def is_empty(self):
26         return len(self.deque) == 0
27
28     def size(self):
29         return len(self.deque)
30 # Example usage:
31 if __name__ == "__main__":
32     my_deque = Deque()
33     my_deque.insert_rear(1)
34     my_deque.insert_rear(2)
35     my_deque.insert_front(0)
36     print("Deque after insertions:", list(my_deque.deque))
37
38     print("Removed from front:", my_deque.remove_front())
39     print("Removed from rear:", my_deque.remove_rear())
40     print("Deque after removals:", list(my_deque.deque))
41     print("Is deque empty?", my_deque.is_empty())
42     print("Size of deque:", my_deque.size())
43
```

The terminal output shows the execution of the code:

```
ment/src/lib/moment/AI &AC.py"
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python
ment/src/lib/moment/AI &AC.py"
Deque after insertions: [0, 1, 2]
Removed from front: 0
Removed from rear: 2
Deque after removals: [1]
Is deque empty? False
Size of deque: 1
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Allows insertion/removal from both ends.
2. Combines features of stack and queue.
3. Efficient **O(1)** operations.
4. Suitable for sliding window problems.
5. Flexible for real-time systems.
6. Useful in palindrome checking and caching.

Task Description #9:Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
 2. Event Registration System – Manage participants in events with quick search and removal.
 3. Library Book Borrowing – Keep track of available books and their due dates.
 4. Bus Scheduling System – Maintain bus routes and stop connections.
 5. Cafeteria Order Queue – Serve students in the order they arrive.
- Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:

Our college wants to develop a Campus Resource Management System that includes:

1. Student Attendance Tracking – Daily log of students entering and exiting campus.

2. Event Registration System – Manage participants with quick search and removal.

3. Library Book Borrowing – Track available books and due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

From the following data structures:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque

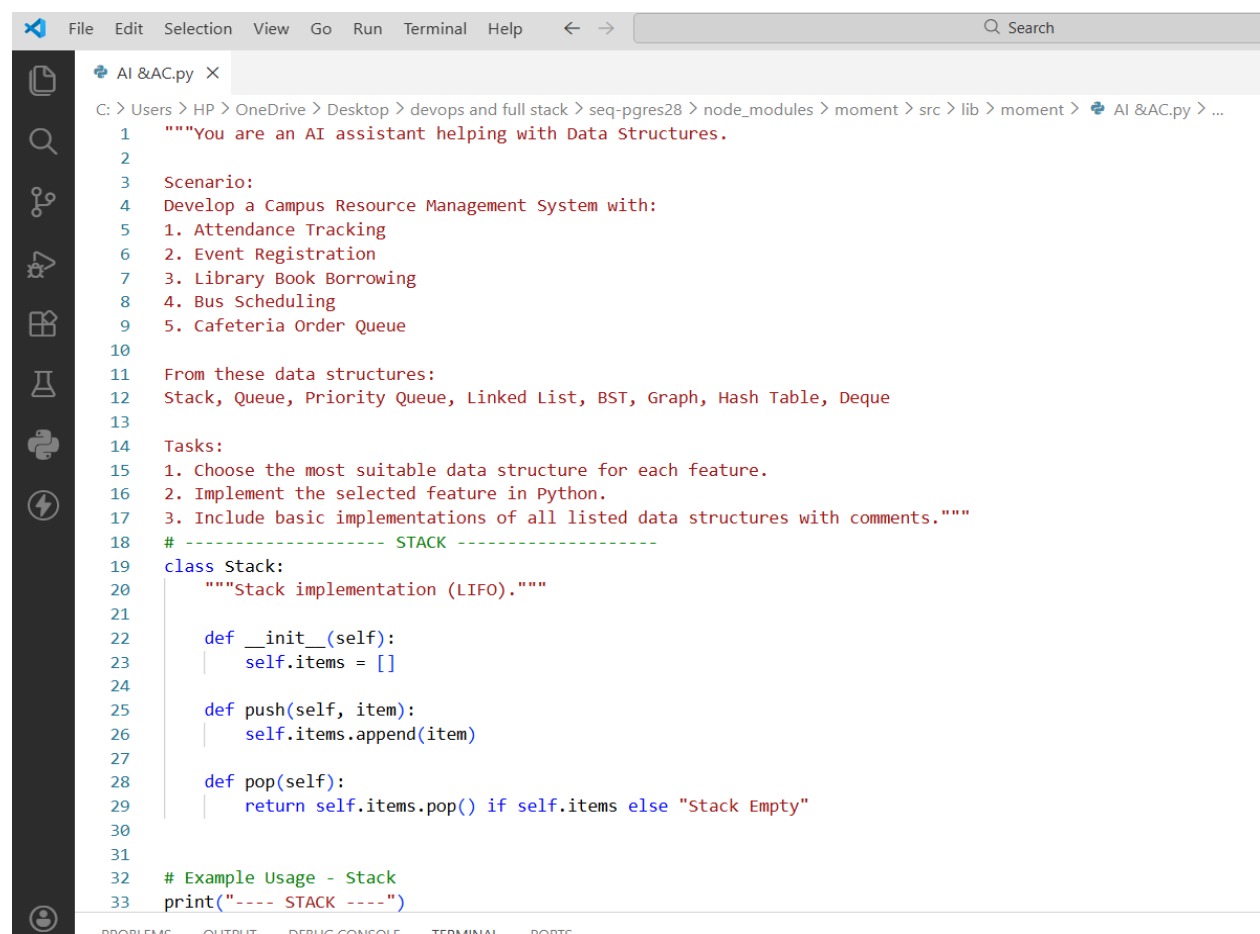
Tasks:

1. Select the most appropriate data structure for each feature.

2. Implement ONE selected feature as a working Python program.

3. Add proper comments and docstrings in the code.

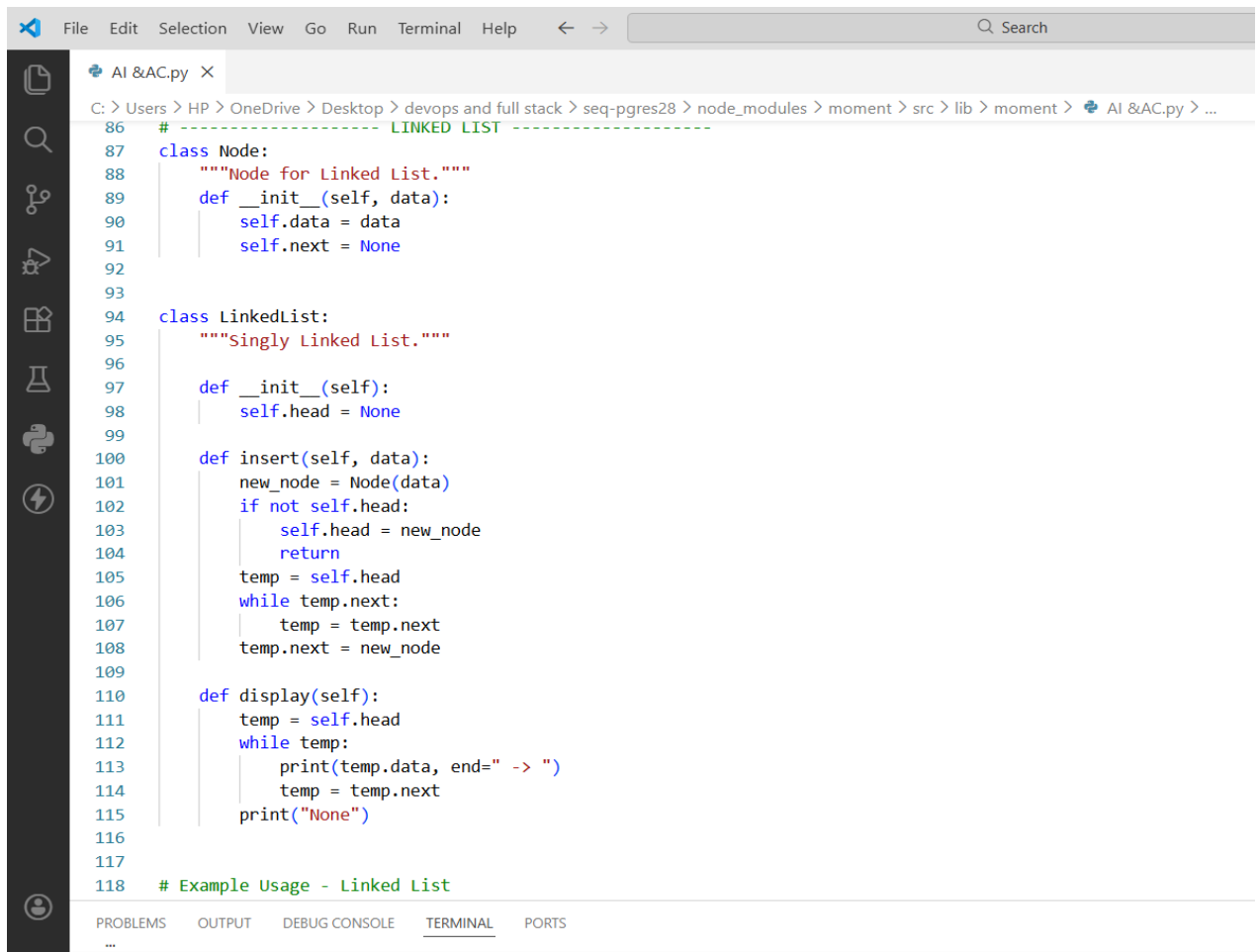
CODE & OUTPUT:

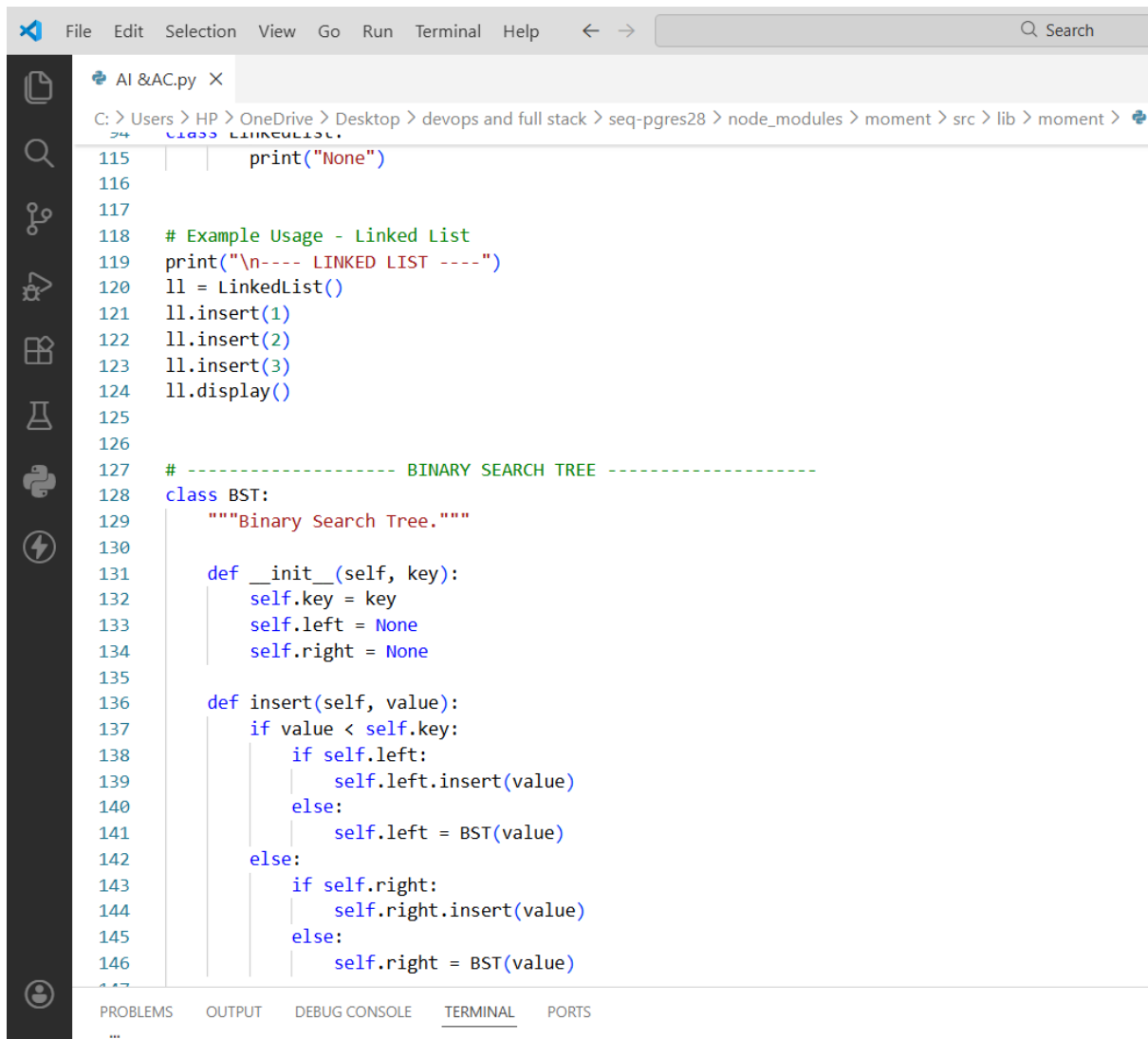


```
File Edit Selection View Go Run Terminal Help < > Search
AI &AC.py X
C:\> Users\HP\OneDrive\Desktop\devops and full stack\seq-pgres28\node_modules\moment\src\lib\moment\AI &AC.py > ...
1  """You are an AI assistant helping with Data Structures.
2
3  Scenario:
4  Develop a Campus Resource Management System with:
5  1. Attendance Tracking
6  2. Event Registration
7  3. Library Book Borrowing
8  4. Bus Scheduling
9  5. Cafeteria Order Queue
10
11 From these data structures:
12 Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque
13
14 Tasks:
15 1. Choose the most suitable data structure for each feature.
16 2. Implement the selected feature in Python.
17 3. Include basic implementations of all listed data structures with comments."""
18 # ----- STACK -----
19 class Stack:
20     """Stack implementation (LIFO)."""
21
22     def __init__(self):
23         self.items = []
24
25     def push(self, item):
26         self.items.append(item)
27
28     def pop(self):
29         return self.items.pop() if self.items else "Stack Empty"
30
31
32 # Example Usage - Stack
33 print("---- STACK ----")
```

```
File Edit Selection View Go Run Terminal Help Search
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
31
32 # Example Usage - Stack
33 print("---- STACK ----")
34 s = Stack()
35 s.push(10)
36 s.push(20)
37 print("Popped:", s.pop())
38
39
40 # ----- QUEUE -----
41 class Queue:
42     """Queue implementation (FIFO)."""
43
44     def __init__(self):
45         self.items = []
46
47     def enqueue(self, item):
48         self.items.append(item)
49
50     def dequeue(self):
51         return self.items.pop(0) if self.items else "Queue Empty"
52
53
54 # Example Usage - Queue
55 print("\n---- QUEUE ----")
56 q = Queue()
57 q.enqueue("Student1")
58 q.enqueue("Student2")
59 print("Dequeued:", q.dequeue())
60
61
62 # ----- PRIORITY QUEUE -----
63 import heapq
```

```
File Edit Selection View Go Run Terminal Help Search
AI &AC.py X
C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment > src > lib > moment > AI &AC.py > ...
61
62 # ----- PRIORITY QUEUE -----
63 import heapq
64
65 class PriorityQueue:
66     """Priority Queue using heap."""
67
68     def __init__(self):
69         self.heap = []
70
71     def enqueue(self, item, priority):
72         heapq.heappush(self.heap, (priority, item))
73
74     def dequeue(self):
75         return heapq.heappop(self.heap)[1] if self.heap else "Empty"
76
77
78 # Example Usage - Priority Queue
79 print("\n---- PRIORITY QUEUE ----")
80 pq = PriorityQueue()
81 pq.enqueue("Low Priority", 2)
82 pq.enqueue("High Priority", 1)
83 print("Dequeued:", pq.dequeue())
84
85
86 # ----- LINKED LIST -----
87 class Node:
88     """Node for Linked List."""
89     def __init__(self, data):
90         self.data = data
91         self.next = None
92
93
```

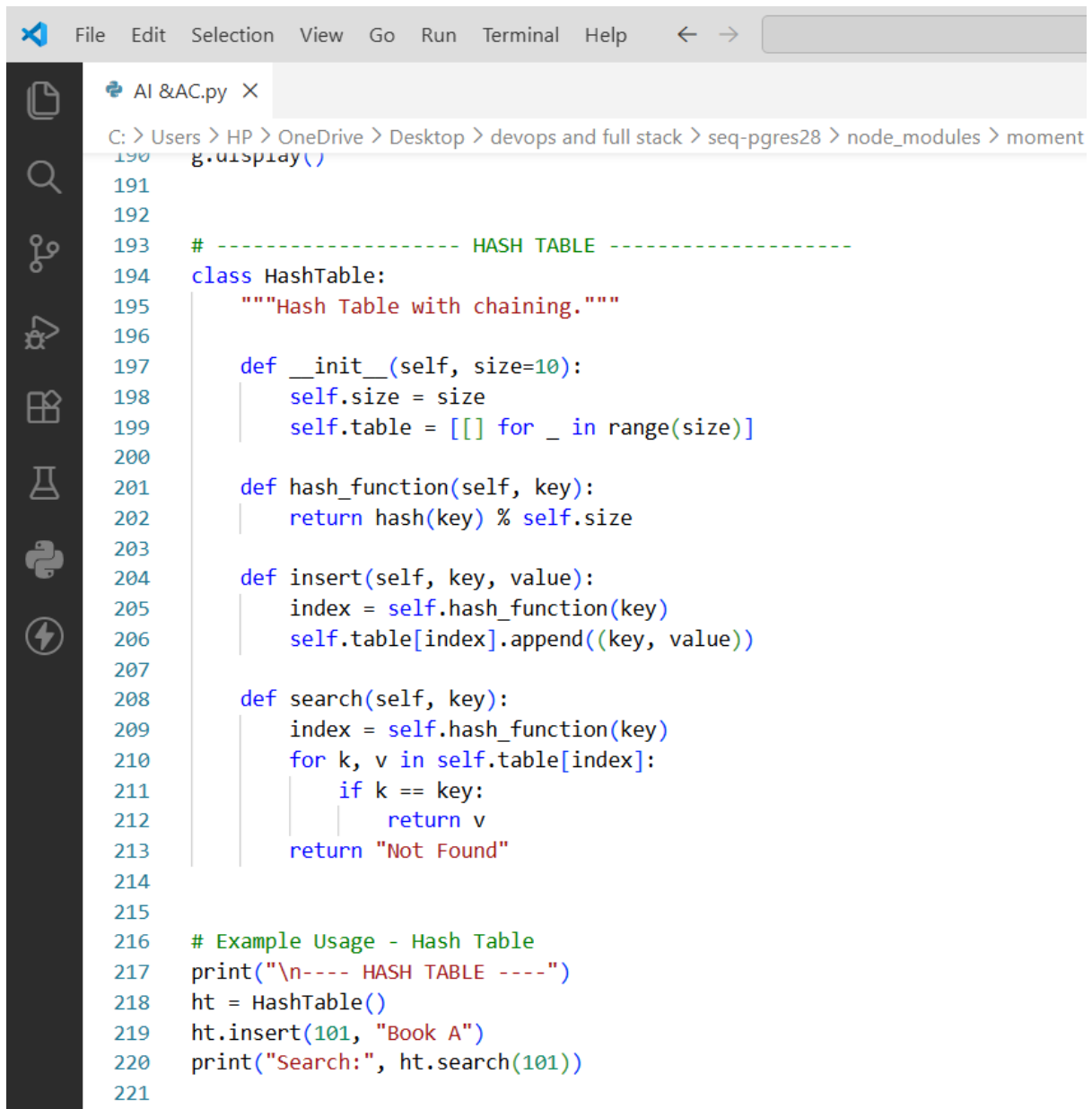




The image shows a Visual Studio Code editor window with a file named `AI & AC.py` open. The file path is `C:\Users\HP\OneDrive\Desktop\devops and full stack\seq-pgres28\node_modules\moment\src\lib\moment`. The code is written in Python and includes a linked list example and a binary search tree class.

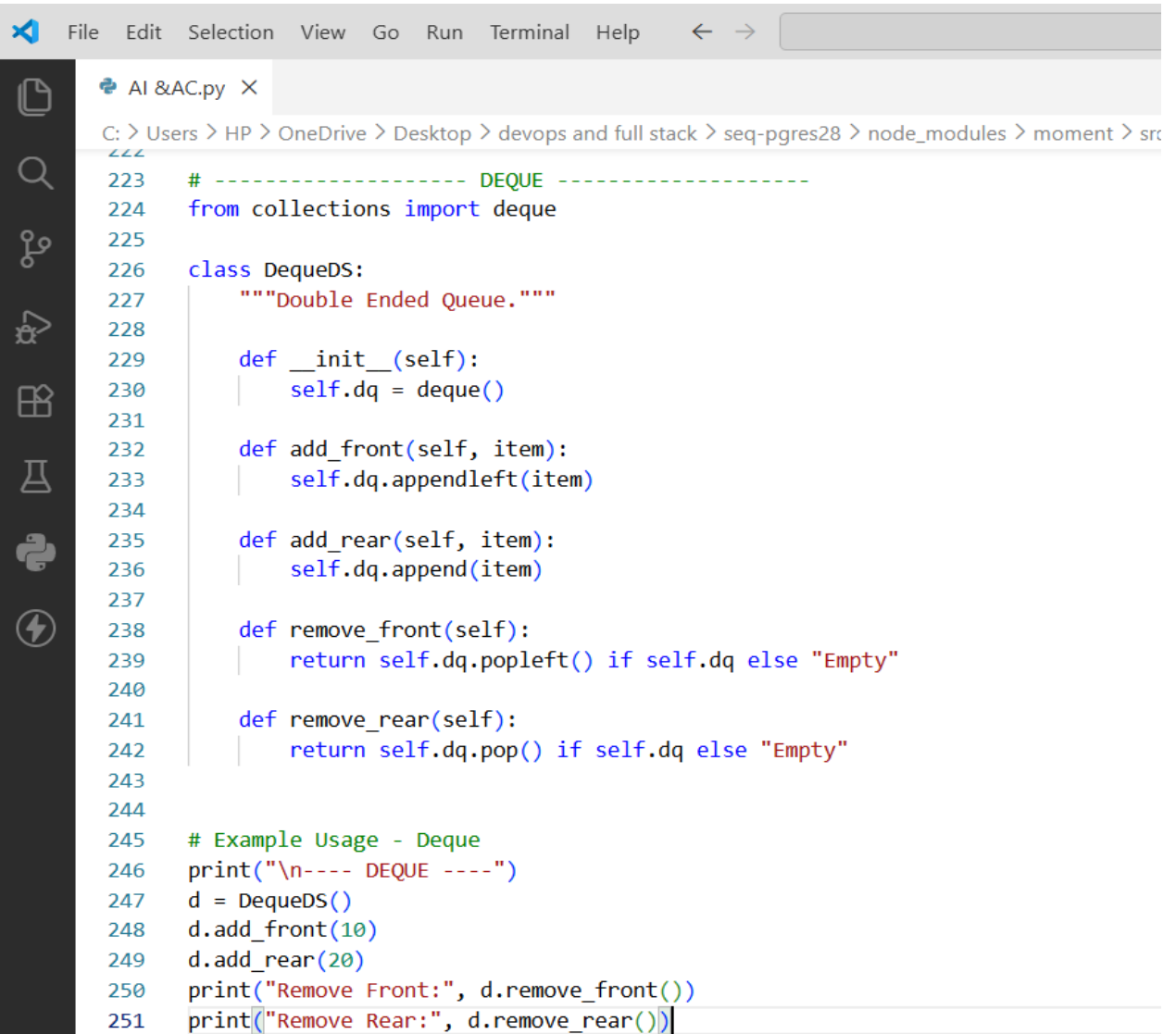
```
115 |         print("None")
116 |
117 |
118 | # Example Usage - Linked List
119 | print("\n---- LINKED LIST ----")
120 | ll = LinkedList()
121 | ll.insert(1)
122 | ll.insert(2)
123 | ll.insert(3)
124 | ll.display()
125 |
126 |
127 | # ----- BINARY SEARCH TREE -----
128 | class BST:
129 |     """Binary Search Tree."""
130 |
131 |     def __init__(self, key):
132 |         self.key = key
133 |         self.left = None
134 |         self.right = None
135 |
136 |     def insert(self, value):
137 |         if value < self.key:
138 |             if self.left:
139 |                 self.left.insert(value)
140 |             else:
141 |                 self.left = BST(value)
142 |         else:
143 |             if self.right:
144 |                 self.right.insert(value)
145 |             else:
146 |                 self.right = BST(value)
```

The editor interface includes a sidebar on the left with icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, and a user profile. The bottom panel shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS, with the TERMINAL tab currently selected.



The image shows a Visual Studio Code editor window with a Python file named 'AI &AC.py'. The file path is 'C: > Users > HP > OneDrive > Desktop > devops and full stack > seq-pgres28 > node_modules > moment'. The code implements a HashTable class with chaining. It includes methods for initialization, hashing, insertion, and searching. An example usage is provided at the bottom, creating a HashTable, inserting 'Book A' at index 101, and searching for it.

```
190 g.display()
191
192
193 # ----- HASH TABLE -----
194 class HashTable:
195     """Hash Table with chaining."""
196
197     def __init__(self, size=10):
198         self.size = size
199         self.table = [[] for _ in range(size)]
200
201     def hash_function(self, key):
202         return hash(key) % self.size
203
204     def insert(self, key, value):
205         index = self.hash_function(key)
206         self.table[index].append((key, value))
207
208     def search(self, key):
209         index = self.hash_function(key)
210         for k, v in self.table[index]:
211             if k == key:
212                 return v
213         return "Not Found"
214
215
216 # Example Usage - Hash Table
217 print("\n---- HASH TABLE ----")
218 ht = HashTable()
219 ht.insert(101, "Book A")
220 print("Search:", ht.search(101))
221
```



The image shows a Visual Studio Code editor window with a Python file named 'AI &AC.py'. The file path is 'C:\Users\HP\OneDrive\Desktop\devops and full stack\seq-pgres28\node_modules\moment\src'. The code defines a 'DequeDS' class that implements a double-ended queue using the 'collections.deque' module. The class has methods for adding and removing elements from both ends. An example usage is provided at the bottom of the file.

```
222
223 # ----- DEQUE -----
224 from collections import deque
225
226 class DequeDS:
227     """Double Ended Queue."""
228
229     def __init__(self):
230         self.dq = deque()
231
232     def add_front(self, item):
233         self.dq.appendleft(item)
234
235     def add_rear(self, item):
236         self.dq.append(item)
237
238     def remove_front(self):
239         return self.dq.popleft() if self.dq else "Empty"
240
241     def remove_rear(self):
242         return self.dq.pop() if self.dq else "Empty"
243
244
245 # Example Usage - Deque
246 print("\n---- DEQUE ----")
247 d = DequeDS()
248 d.add_front(10)
249 d.add_rear(20)
250 print("Remove Front:", d.remove_front())
251 print("Remove Rear:", d.remove_rear())
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python39\python.exe C:\Users\HP\AppData\Local\Programs\Python\Python39\Scripts\pip.exe install moment/src/lib/moment/AI & AC.py"
---- STACK ----
Popped: 20

---- QUEUE ----
Dequeued: Student1

---- PRIORITY QUEUE ----
Dequeued: High Priority

---- LINKED LIST ----
1 -> 2 -> 3 -> None

---- BST ----
20 30 50 70
---- GRAPH ----
A -> ['B']
B -> []

---- HASH TABLE ----
Search: Book A

---- DEQUE ----
Remove Front: 10
Remove Rear: 20
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

Data Structure	Justification
Stack	1. Follows LIFO principle. 2. Fast push and pop ($O(1)$). 3. Used in recursion and undo operations.
Queue	1. Follows FIFO principle. 2. Maintains order of processing. 3. Used in scheduling systems.
Priority Queue	1. Processes elements by priority. 2. Efficient using heap. 3. Used in task scheduling.
Linked List	1. Dynamic size.

	2. Easy insertion and deletion. 3. Flexible memory usage.
Binary Search Tree (BST)	1. Maintains sorted data. 2. Efficient searching. 3. In-order traversal gives sorted output.
Graph	1. Represents connections between nodes. 2. Used in route planning. 3. Models real-world networks.
Hash Table	1. Fast lookup ($O(1)$). 2. Efficient key-value storage. 3. Quick insertion and deletion.
Deque	1. Insert/remove from both ends. 2. Combines stack and queue features. 3. Efficient operations.

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from

the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

From these data structures:

Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque

Tasks:

1. Choose the most suitable data structure for each feature.
2. Implement one selected feature as a working Python program.
3. Add proper comments and docstrings.

CODE & OUTPUT:

```
File Edit Selection View Go Run Terminal Help Search
AI &AC.py X
C:\Users\HP> OneDrive\ Desktop\ devops and full stack\ seq-pgres28\ node_modules\ moment\ src\ lib\ moment\ AI &AC.py > ...
1 """
2 Scenario:
3 An e-commerce company wants to build a Smart Online Shopping System with:
4
5 1. Shopping Cart Management Add and remove products dynamically.
6 2. Order Processing System Orders processed in the order they are placed.
7 3. Top-Selling Products Tracker Products ranked by sales count.
8 4. Product Search Engine Fast lookup of products using product ID.
9 5. Delivery Route Planning Connect warehouses and delivery locations.
10
11 From these data structures:
12 Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque
13
14 Tasks:
15 1. Choose the most suitable data structure for each feature.
16 2. Implement one selected feature as a working Python program.
17 3. Add proper comments and docstrings."""
18 class ProductSearch:
19     """
20     Product Search Engine using Hash Table.
21     Provides fast lookup using Product ID.
22     """
23
24     def __init__(self, size=10):
25         """Initialize hash table with given size."""
26         self.size = size
27         self.table = [[] for _ in range(size)]
28
```

```
File Edit Selection View Go Run Terminal Help Search
AI &AC.py X
C:\Users\HP> HP> OneDrive\ Desktop\ devops and full stack\ seq-pgres28\ node_modules\ moment\ src\ lib\ moment\ AI &AC.py > ...
18 class ProductSearch:
24     def __init__(self, size=10):
27         self.table = [[] for _ in range(size)]
28
29     def hash_function(self, product_id):
30         """Generate index using hash function."""
31         return hash(product_id) % self.size
32
33     def insert(self, product_id, product_name):
34         """
35         Insert a product into the system.
36         :param product_id: Unique product ID
37         :param product_name: Name of the product
38         """
39         index = self.hash_function(product_id)
40         self.table[index].append((product_id, product_name))
41
42     def search(self, product_id):
43         """
44         Search product by ID.
45         Returns product name if found.
46         """
47         index = self.hash_function(product_id)
48         for pid, pname in self.table[index]:
49             if pid == product_id:
50                 return pname
51         return "Product Not Found"
52
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

The screenshot shows the Visual Studio Code interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. A search bar is located at the top right. The left sidebar contains icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, Accounts, and Settings. The main editor area displays the file AI & AC.py with the following Python code:

```
18 class ProductSearch:  
42     def search(self, product_id):  
43  
44         Search product by ID.  
45         Returns product name if found.  
46         """"  
47         index = self.hash_function(product_id)  
48         for pid, pname in self.table[index]:  
49             if pid == product_id:  
50                 return pname  
51         return "Product Not Found"  
52  
53  
54 # Example Usage  
55 store = ProductSearch()  
56  
57 store.insert(101, "Laptop")  
58 store.insert(102, "Mobile")  
59  
60 print("Search Result:", store.search(101))
```

The bottom panel shows the TERMINAL tab with the following output:

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Py  
les/moment/src/lib/moment/AI &AC.py"  
Search Result: Laptop  
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Allows easy add and remove of products dynamically. No shifting of elements needed.
Order Processing System	Queue	Follows FIFO order. Orders are processed in the same order they are placed.

Top-Selling Products Tracker	Priority Queue	Maintains products based on highest sales. Quickly retrieves top-selling product.	
Product Search Engine	Hash Table	Provides fast product lookup using product ID. Search time is very efficient ($O(1)$).	
Delivery Route Planning	Graph	Represents locations as nodes and routes as edges. Helps find optimal delivery paths.	